

PASOS PREVIOS

Creen una carpeta nueva llamada DATASET_PULMONES en la raíz de su "Mi Unidad".
Entren a esa carpeta y arrastren directamente las carpetas train y test

Paso 1: Conectar el Drive en Colab

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Paso 2: extraendo la data set del drive y normalizando la data set

```
import tensorflow as tf
import os

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

# Ruta a tu nueva estructura unificada
ruta_train = '/content/drive/MyDrive/DATASET_PULMONES/train'
ruta_test = '/content/drive/MyDrive/DATASET_PULMONES/test'

print("Cargando Dataset de Entrenamiento (3 Clases)...")
train_ds = tf.keras.utils.image_dataset_from_directory(
    ruta_train,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="training",
    seed=42
)

print("\nCargando Dataset de Validación...")
val_ds = tf.keras.utils.image_dataset_from_directory(
    ruta_train,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    validation_split=0.2,
    subset="validation",
    seed=42
)

print("\nCargando Dataset de Pruebas (Test)...")
test_ds = tf.keras.utils.image_dataset_from_directory(
    ruta_test,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False # Importante para la Matriz de Confusión
)

# Normalización
normalization_layer = tf.keras.layers.Rescaling(1./255)
train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))
```

Cargando Dataset de Entrenamiento (3 Clases)...
Found 6096 files belonging to 3 classes.
Using 4877 files for training.

Cargando Dataset de Validación...
Found 6096 files belonging to 3 classes.
Using 1219 files for validation.

Cargando Dataset de Pruebas (Test)...
Found 674 files belonging to 3 classes.

Paso 3: Definir la Red Neuronal

```
from tensorflow.keras import layers, models

# Usamos MobileNetV2 como base (Transfer Learning)
base_model = tf.keras.applications.MobileNetV2(input_shape=(224, 224, 3), include_top=False, weights='imagenet')
base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax') # CAMBIO CLAVE: 3 clases
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_2 (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 3)	387

Total params: 2,422,339 (9.24 MB)
Trainable params: 164,355 (642.01 KB)
Non-trainable params: 2,257,984 (8.61 MB)

Paso 4: Este es el entramiento.

```
# Definimos 10 épocas de entrenamiento (ciclos completos de aprendizaje)
EPOCHS = 10

print("Iniciando el proceso de adquisición de Experiencia (E) con Aprendizaje Supervisado...")
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
print("\nEntrenamiento finalizado con éxito!")
```

Iniciando el proceso de adquisición de Experiencia (E) con Aprendizaje Supervisado...

Epoch 1/10
153/153 — 103s 593ms/step - accuracy: 0.9028 - loss: 0.2575 - val_accuracy: 0.9549 - val_loss: 0.1377

Epoch 2/10
153/153 — 66s 429ms/step - accuracy: 0.9598 - loss: 0.1123 - val_accuracy: 0.9631 - val_loss: 0.1064

Epoch 3/10
153/153 — 69s 448ms/step - accuracy: 0.9705 - loss: 0.0846 - val_accuracy: 0.9598 - val_loss: 0.1151

Epoch 4/10
153/153 — 72s 473ms/step - accuracy: 0.9744 - loss: 0.0674 - val_accuracy: 0.9672 - val_loss: 0.0836

Epoch 5/10
153/153 — 66s 433ms/step - accuracy: 0.9815 - loss: 0.0515 - val_accuracy: 0.9664 - val_loss: 0.0834

Epoch 6/10

```

153/153 ————— 69s 450ms/step - accuracy: 0.9820 - loss: 0.0529 - val_accuracy: 0.9721 - val_loss: 0.0748
Epoch 7/10
153/153 ————— 66s 428ms/step - accuracy: 0.9861 - loss: 0.0361 - val_accuracy: 0.9598 - val_loss: 0.0877
Epoch 8/10
153/153 ————— 68s 442ms/step - accuracy: 0.9869 - loss: 0.0333 - val_accuracy: 0.9680 - val_loss: 0.0727
Epoch 9/10
153/153 ————— 73s 477ms/step - accuracy: 0.9873 - loss: 0.0347 - val_accuracy: 0.9729 - val_loss: 0.0786
Epoch 10/10
153/153 ————— 74s 481ms/step - accuracy: 0.9916 - loss: 0.0287 - val_accuracy: 0.9705 - val_loss: 0.0742

¡Entrenamiento finalizado con éxito!

```

Paso 5: Código para Gráficas de Rendimiento

```

import matplotlib.pyplot as plt

# Extraemos los datos del historial de entrenamiento
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(EPOCHS)

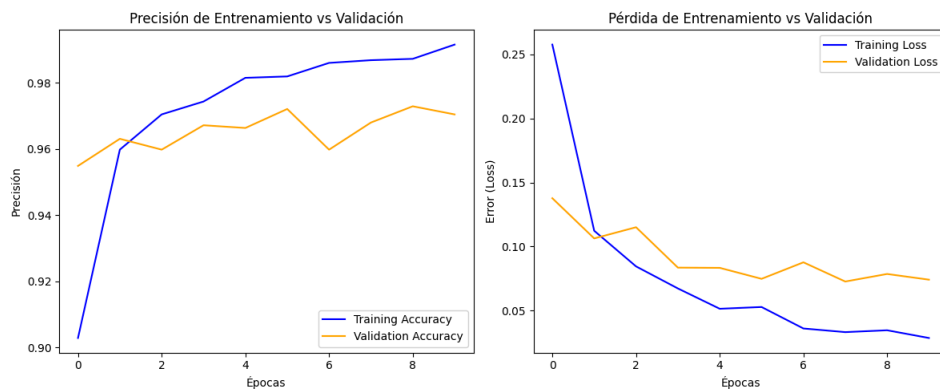
# Dibujamos las gráficas de rendimiento metodológico
plt.figure(figsize=(12, 5))

# Gráfica 1: Precisión (Accuracy)
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy', color='blue')
plt.plot(epochs_range, val_acc, label='Validation Accuracy', color='orange')
plt.title('Precisión de Entrenamiento vs Validación')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend(loc='lower right')

# Gráfica 2: Pérdida (Loss)
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss', color='blue')
plt.plot(epochs_range, val_loss, label='Validation Loss', color='orange')
plt.title('Pérdida de Entrenamiento vs Validación')
plt.xlabel('Épocas')
plt.ylabel('Error (Loss)')
plt.legend(loc='upper right')

plt.tight_layout()
plt.show()

```



Paso 6: Evaluación Final con el Set de Datos de Prueba (Test)

```

print("Evaluando el rendimiento final en el conjunto de datos de Test...")
results = model.evaluate(test_ds)
print(f"\nPrecisión final en Test: {results[1]*100:.2f}%")
print(f"Pérdida final en Test: {results[0]:.4f}")

Evaluando el rendimiento final en el conjunto de datos de Test...
20/20 ————— 170s 8s/step - accuracy: 0.8462 - loss: 0.7200

Precisión final en Test: 84.62%
Pérdida final en Test: 0.7200

```

Paso 7: Matriz de Confusión (Análisis de Falsos Positivos/Negativos)

```

import numpy as np
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

nombres_clases = train_ds.class_names # ['COVID19', 'NORMAL', 'PNEUMONIA']

y_pred = []
y_true = []

for images, labels in test_ds:
    preds = model.predict(images)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())

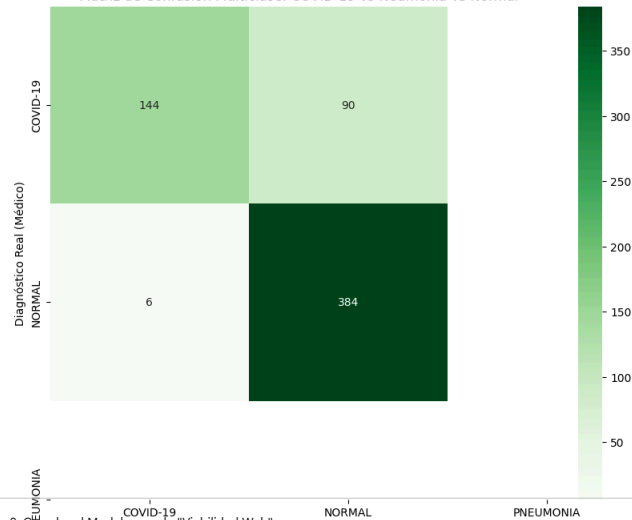
# Dibujar Matriz
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=nombres_clases, yticklabels=nombres_clases)
plt.xlabel('Predicción IA')
plt.ylabel('Realidad Médica')
plt.title('Resultados Finales del Proyecto')
plt.show()

print(classification_report(y_true, y_pred, target_names=nombres_clases))

```

1/1 0s 99ms/step
1/1 0s 112ms/step
1/1 0s 143ms/step
1/1 0s 117ms/step
1/1 0s 126ms/step
1/1 0s 116ms/step
1/1 0s 116ms/step
1/1 0s 104ms/step
1/1 0s 124ms/step
1/1 0s 104ms/step
1/1 0s 118ms/step
1/1 0s 107ms/step
1/1 0s 136ms/step
1/1 0s 110ms/step
1/1 0s 112ms/step
1/1 0s 144ms/step
1/1 0s 147ms/step
1/1 0s 111ms/step
1/1 0s 99ms/step
1/1 0s 60ms/step

Matriz de Confusión Multiclase: COVID-19 vs Neumonía vs Normal



Paso 8: Guardar el Modelo para la "Viabilidad Web"

```
# Guardamos el modelo en formato estándar H5
model.save('modelo_neumonia_deeplearning.h5')

from google.colab import files
files.download('modelo_neumonia_deeplearning.h5')
print("¡Modelo descargado! Este archivo es el que usarías en tu página web.")
```

El Toque Final: Prueba con una Imagen Real

```
!python3 /usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py in classification_report(y_true, y_pred, labels, target_names, sample_weight, digits, output_dict, zero_division)

from google.colab import files
from tensorflow.keras.preprocessing import image

print("Sube una radiografía para que la IA la analice:")
uploaded = files.upload()

for fn in uploaded.keys():
    path = fn
    img = image.load_img(path, target_size=(224, 224))
    x = image.img_to_array(img) / 255.0
    x = np.expand_dims(x, axis=0)

    classes = model.predict(x)
    resultado = np.argmax(classes[0])

    print(f"\nResultado del Análisis: {nombres_clases[resultado]}")
    print(f"Confianza: {np.max(classes[0])*100:.2f}%")
```

```
import os
import shutil
import numpy as np
import tensorflow as tf
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Creamos la estructura temporal
os.makedirs('/content/temp_test/COVID-19', exist_ok=True)
os.makedirs('/content/temp_test/NORMAL', exist_ok=True)
os.makedirs('/content/temp_test/PNEUMONIA', exist_ok=True)

# 2. Rutas de origen (ajustadas a tu Drive)
ruta_covid = '/content/drive/MyDrive/DATASET_PULMONES/imagenes_covid19'
ruta_normal = '/content/drive/MyDrive/DATASET_PULMONES/test/NORMAL'
ruta_pneumonia = '/content/drive/MyDrive/DATASET_PULMONES/test/PNEUMONIA'

def copiar_contenido(origen, destino):
    if os.path.exists(origen):
        archivos = [f for f in os.listdir(origen) if os.path.isfile(os.path.join(origen, f))]
        for file in archivos[:100]:
            shutil.copy(os.path.join(origen, file), os.path.join(destino, file))

copiar_contenido(ruta_covid, '/content/temp_test/COVID-19')
copiar_contenido(ruta_normal, '/content/temp_test/NORMAL')
copiar_contenido(ruta_pneumonia, '/content/temp_test/PNEUMONIA')

# 3. Cargamos el dataset y GUARDAMOS los nombres antes del error
test_raw = tf.keras.utils.image_dataset_from_directory(
    '/content/temp_test',
    image_size=(224, 224),
    batch_size=32,
    shuffle=False
)

# AQUÍ ESTÁ EL TRUCO: Guardamos los nombres antes de normalizar
nombres_clases = test_raw.class_names

# Ahora sí normalizamos
normalization_layer = tf.keras.layers.Rescaling(1./255)
test_unificado = test_raw.map(lambda x, y: (normalization_layer(x), y))

# 4. Generamos la Matriz de Confusión
y_pred = []
y_true = []

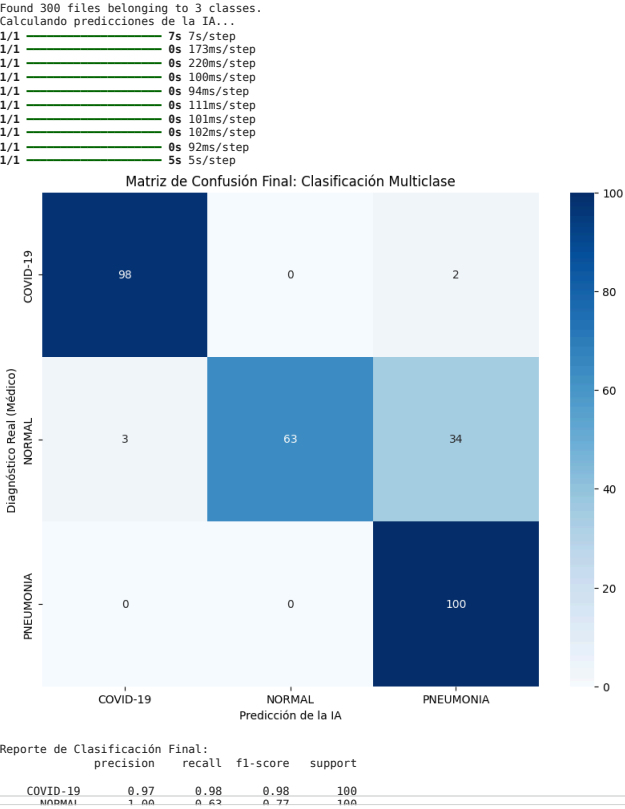
print("Calculando predicciones de la IA...")
for images, labels in test_unificado:
    preds = model.predict(images)
    y_pred.extend(np.argmax(preds, axis=1))

y_true.extend(labels)
```

```
y_true.extend(labels.numpy())

# Visualización
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=nombres_clases, yticklabels=nombres_clases)
plt.xlabel('Predicción de la IA')
plt.ylabel('Diagnóstico Real (Médico)')
plt.title('Matriz de Confusión Final: Clasificación Multiclase')
plt.show()

print("\nReporte de Clasificación Final:")
print(classification_report(y_true, y_pred, target_names=nombres_clases))
```



```
import numpy as np
from google.colab import files
from tensorflow.keras.utils import load_img, img_to_array

# Los nombres ahora coinciden exactamente con tus carpetas de Drive
nombres_clases = ['COVID19', 'NORMAL', 'PNEUMONIA']

print("Sube una radiografía para el diagnóstico final:")
uploaded = files.upload()

for fn in uploaded.keys():
    img = load_img(fn, target_size=(224, 224))
    x = img_to_array(img) / 255.0
    x = np.expand_dims(x, axis=0)

    preds = model.predict(x)
    resultado_idx = np.argmax(preds[0])

    print(f"\n--- REPORTE MÉDICO DE IA ---")
    print(f"Diagnóstico sugerido: {nombres_clases[resultado_idx]}")
    print(f"Confianza del motor: {preds[0][resultado_idx]*100:.2f}%")
    print(f"-----")

    # Esto es para tu informe: mostrar la incertidumbre (Semana 5)
    print("\nProbabilidades detalladas:")
    for i, clase in enumerate(nombres_clases):
        print(f" - {clase}: {preds[0][i]*100:.2f}%")
```

Sube una radiografía para el diagnóstico final:

[Elegir archivos](#) No se eligió ningún archivo Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving NORMAL2-IM-0316-0001.jpeg to NORMAL2-IM-0316-0001.jpeg
1/1 ————— 0s 72ms/step

--- REPORTE MÉDICO DE IA ---
Diagnóstico sugerido: PNEUMONIA
Confianza del motor: 99.22%

Probabilidades detalladas:
- COVID19: 0.01%
- NORMAL: 0.76%
- PNEUMONIA: 99.22%